

International Airport Database

Entity–Relationship Diagram and Schema Description

Course: Databases

Laboratory work: 1 — ERD diagram

Prepared by: Imangali Yerden

Source specification: International Airport database brief provided with the assignment

1. Solution overview

The proposed relational model manages airports, airlines, scheduled and actual flight operations, passenger bookings, boarding passes, baggage registration, baggage checks, passenger security checks, and booking-change history. The design uses surrogate numeric primary keys and explicit foreign keys to maintain referential integrity. Timestamps record creation and modification events for operational records.

The diagram uses crow's-foot notation. A mandatory relationship endpoint is shown with `||`, an optional single endpoint with `o|`, and an optional collection endpoint with `o{`. Therefore, the diagram distinguishes required parent ownership from optional child records.

2. Entity–relationship diagram

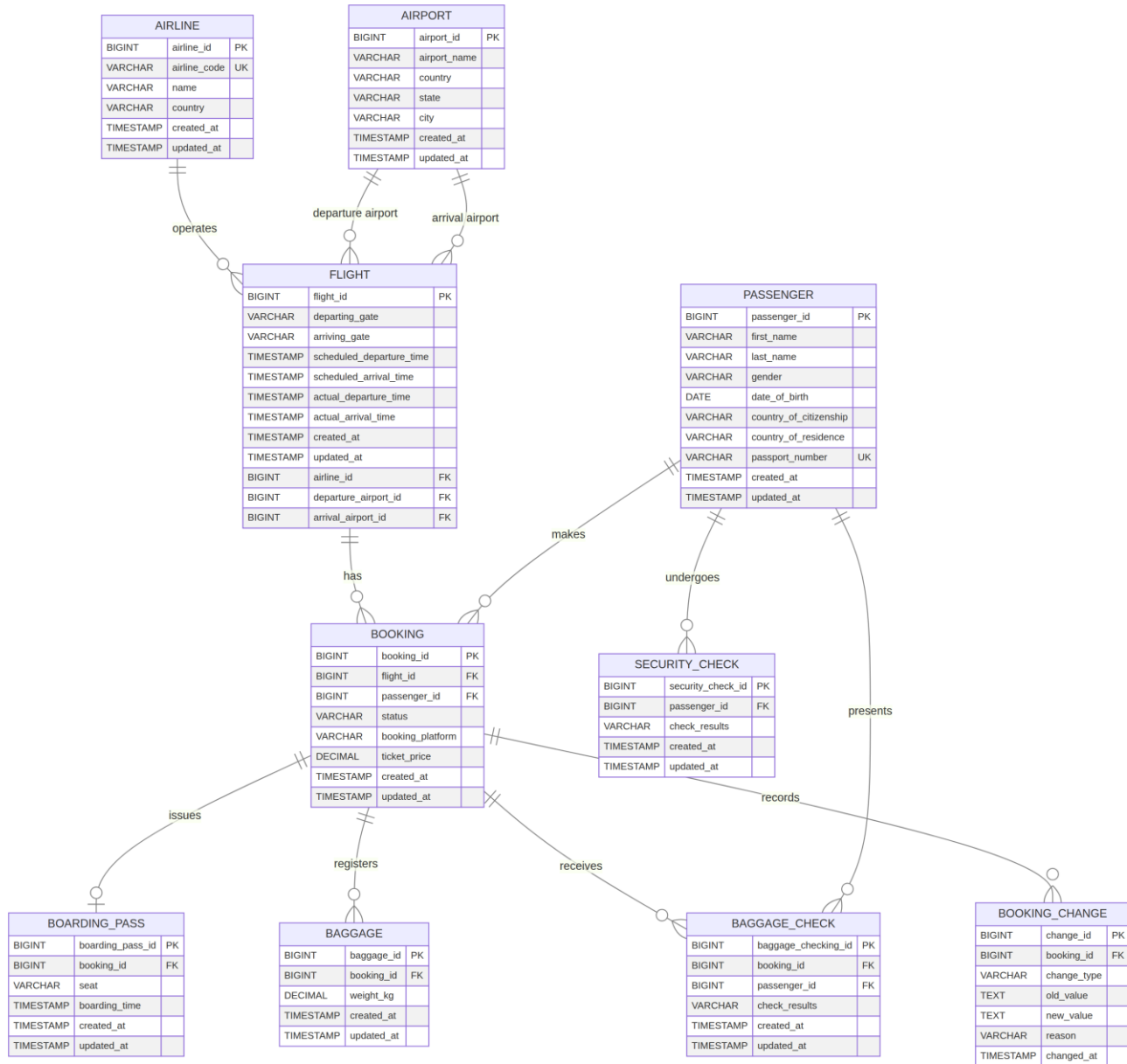


Figure 1. Complete crow's-foot ERD for the International Airport database.

3. Entity and attribute specification

The following table documents every entity shown in the diagram. 'PK' means primary key, 'FK' means foreign key, 'UQ' means unique, and 'NN' means required ('NOT NULL'). Optional attributes are left without the 'NN' marker.

Entity	Attribute	SQL type	Constraint / meaning
Airport	airport_id	BIGINT	PK
Airport	airport_name	VARCHAR	NN
Airport	country	VARCHAR	NN
Airport	state	VARCHAR	Optional administrative region
Airport	city	VARCHAR	NN
Airport	created_at	TIMESTAMP	NN
Airport	updated_at	TIMESTAMP	NN

Airline	airline_id	BIGINT	PK
Airline	airline_code	VARCHAR	UQ, NN
Airline	name	VARCHAR	NN
Airline	country	VARCHAR	NN
Airline	created_at	TIMESTAMP	NN
Airline	updated_at	TIMESTAMP	NN
Flight	flight_id	BIGINT	PK
Flight	departing_gate	VARCHAR	Optional gate identifier
Flight	arriving_gate	VARCHAR	Optional gate identifier
Flight	scheduled departure time	TIMESTAMP	NN
Flight	scheduled arrival time	TIMESTAMP	NN
Flight	actual departure time	TIMESTAMP	Optional until departure occurs
Flight	actual arrival time	TIMESTAMP	Optional until arrival occurs
Flight	created_at	TIMESTAMP	NN
Flight	updated_at	TIMESTAMP	NN
Flight	airline_id	BIGINT	FK → Airline.airline_id, NN
Flight	departure airport_id	BIGINT	FK → Airport.airport_id, NN
Flight	arrival airport_id	BIGINT	FK → Airport.airport_id, NN
Passenger	passenger_id	BIGINT	PK
Passenger	first name	VARCHAR	NN
Passenger	last name	VARCHAR	NN
Passenger	gender	VARCHAR	NN
Passenger	date of birth	DATE	NN
Passenger	country of citizenship	VARCHAR	NN
Passenger	country of residence	VARCHAR	NN
Passenger	passport number	VARCHAR	UQ, NN
Passenger	created_at	TIMESTAMP	NN
Passenger	updated_at	TIMESTAMP	NN
Booking	booking_id	BIGINT	PK
Booking	flight_id	BIGINT	FK → Flight.flight_id, NN
Booking	passenger_id	BIGINT	FK → Passenger.passenger_id, NN
Booking	status	VARCHAR	NN; use controlled status values
Booking	booking_platform	VARCHAR	NN
Booking	ticket_price	DECIMAL(10,2)	NN
Booking	created_at	TIMESTAMP	NN
Booking	updated_at	TIMESTAMP	NN
BookingChange	change_id	BIGINT	PK
BookingChange	booking_id	BIGINT	FK → Booking.booking_id, NN
BookingChange	change_type	VARCHAR	NN
BookingChange	old_value	TEXT	Previous value, when applicable
BookingChange	new_value	TEXT	Replacement value, when applicable
BookingChange	reason	VARCHAR	Optional explanation
BookingChange	changed_at	TIMESTAMP	NN
BoardingPass	boarding_pass_id	BIGINT	PK
BoardingPass	booking_id	BIGINT	FK → Booking.booking_id, NN; UQ
BoardingPass	seat	VARCHAR	NN
BoardingPass	boarding_time	TIMESTAMP	Optional until boarding
BoardingPass	created_at	TIMESTAMP	NN
BoardingPass	updated_at	TIMESTAMP	NN
Baggage	baggage_id	BIGINT	PK
Baggage	booking_id	BIGINT	FK → Booking.booking_id, NN
Baggage	weight kg	DECIMAL(8,2)	NN; non-negative value
Baggage	created_at	TIMESTAMP	NN
Baggage	updated_at	TIMESTAMP	NN
BaggageCheck	baggage_checking_id	BIGINT	PK
BaggageCheck	booking_id	BIGINT	FK → Booking.booking_id, NN
BaggageCheck	passenger_id	BIGINT	FK → Passenger.passenger_id, NN
BaggageCheck	check_results	VARCHAR	NN
BaggageCheck	created_at	TIMESTAMP	NN
BaggageCheck	updated_at	TIMESTAMP	NN
SecurityCheck	security_check_id	BIGINT	PK
SecurityCheck	passenger_id	BIGINT	FK →

			Passenger.passenger_id, NN
SecurityCheck	check_results	VARCHAR	NN
SecurityCheck	created_at	TIMESTAMP	NN
SecurityCheck	updated_at	TIMESTAMP	NN

4. Relationships and participation constraints

Relationship	Cardinality	Participation and interpretation
Airline operates Flight	1 : 0..*	Every flight belongs to exactly one airline. An airline may operate no flights or many flights.
Airport departs Flight	1 : 0..*	Every flight has exactly one departure airport. An airport may have no departing flights or many.
Airport receives Flight	1 : 0..*	Every flight has exactly one arrival airport. An airport may have no arriving flights or many.
Flight has Booking	1 : 0..*	Every booking is for exactly one flight. A flight may have no bookings or many.
Passenger makes Booking	1 : 0..*	Every booking is made by exactly one passenger. A passenger may have no bookings or many.
Booking issues BoardingPass	1 : 0..1	A boarding pass belongs to exactly one booking. A booking may not yet have a boarding pass or may have one; `UNIQUE(booking_id)` enforces this one-to-zero-or-one rule.
Booking registers Baggage	1 : 0..*	Each baggage record belongs to exactly one booking. A booking may have no baggage or multiple baggage records.
Booking receives BaggageCheck	1 : 0..*	Each baggage-check record references exactly one booking. A booking may have no checks or many.
Passenger presents BaggageCheck	1 : 0..*	Each baggage-check record references exactly one passenger. A passenger may have no checks or many.
Passenger undergoes SecurityCheck	1 : 0..*	Each security-check record references exactly one passenger. A passenger may have no checks or many.
Booking records BookingChange	1 : 0..*	Each change-history record belongs to exactly one booking. A booking may have no changes or many.

5. Integrity and modeling assumptions

The two airport foreign keys in `Flight` represent different roles: `departure\_airport\_id` identifies where the flight departs, while `arrival\_airport\_id` identifies where it arrives. Both are mandatory because a scheduled flight requires both endpoints.

A booking is the associative record between a passenger and a flight. This design permits one passenger to book many flights and one flight to have many passengers without storing a repeating passenger list inside `Flight`.

The specification requires booking changes to be stored separately. The `BookingChange` entity therefore provides an auditable history of changed values. `old\_value` and `new\_value` are stored as text because different booking fields may have different underlying types; a production implementation could instead use typed change columns or a JSON audit payload.

The source brief mentions aircraft information and frequent-flyer programs in a general discussion, but it provides no attributes or relationship rules for those subjects. They are consequently outside the modeled scope rather than being invented in the ERD.

The source also requires `passenger\_id` on `BaggageCheck`, so it is retained even though the booking already identifies the passenger. An implementation should validate that the baggage-check passenger matches the passenger associated with the referenced booking.

6. Notation legend

Symbol / marker	Meaning
PK	Primary key; uniquely identifies an entity record
FK	Foreign key; references a primary key in another entity
UQ	Unique constraint; duplicate values are not permitted
NN	Required value; implemented as `NOT NULL`
	Exactly one related record
O	Zero or one related record
`o{`	Zero or many related records
`BIGINT`, `VARCHAR`, `DATE`, `TIMESTAMP`, `DECIMAL`, `TEXT`	Suggested SQL data types